

Gymnázium J. K. Tyla



Vzrůst Agentického AI: úvod do agentického inženýrství a implementace software factory

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce představuje agentickou umělou inteligenci jako poslední krok v přechodu od konverzační asistence k delegovanému vývoji a vymezuje pojem agentického inženýrství, který dosud nemá v češtině ustálené označení. Ukazuje, že jádrem této disciplíny není schopnost jazykového modelu, ale návrh systému, v němž model pracuje, a že pojem software factory, zavedený již v roce 1968, zažívá právě prostřednictvím agentů znovu. Z výzkumné otázky, zda je řízená autonomie dosažitelná bez vlastního harnessu, vychází hypotéza, která je v praktické části ověřována. Ta popisuje DarkFactory, záměrně jednoduchou produkční softwareovou továrnu, v níž je GitHub vývojovým prostředím a místem trvalého stavu, GitHub Actions výpočetním prostředím, kontejner izolačním prostředím a pythonovský řadič ve spolupráci s produkčními harnessy realizuje interpretaci, plánování, implementaci, revizi a integraci změn. Hypotéza se potvrzuje: autonomie je dosažitelná i bez vlastního harnessu, její struktura je vůči volbě nástroje invariantní, její kvalita však zůstává omezena kvalitou cizí smyčky. Význam práce spočívá v tom, že ukazuje cestu k řízenému předávání práce bez nutnosti psát si vlastní nástroj, což je právě podmínka, která dnes širšímu nasazení brání.

Klíčová slova: coding agents; software factory; AI; Agentic Engineering; harness

Abstract

This thesis presents agentic artificial intelligence as the latest step in the transition from conversational assistance to delegated development, and establishes the term agentic engineering, which has no settled Czech equivalent. It argues that the core of the discipline is not the capability of a language model but the design of the system in which the model operates, and that software factory, a concept introduced in 1968, is being revived precisely through agents. The research question — whether controlled autonomy is reachable without an in-house harness — yields a hypothesis that the practical part then tests. That part describes DarkFactory, a deliberately simple production software factory in which GitHub is the development environment and the durable state, GitHub Actions the execution environment, a container the isolation layer, and a Python runner working with production harnesses performs intake, planning, implementation, review and integration. The hypothesis is confirmed: autonomy is reachable without an in-house harness and its structure is invariant to the choice of tool, while its quality remains bounded by the quality of a third-party loop the author does not control.

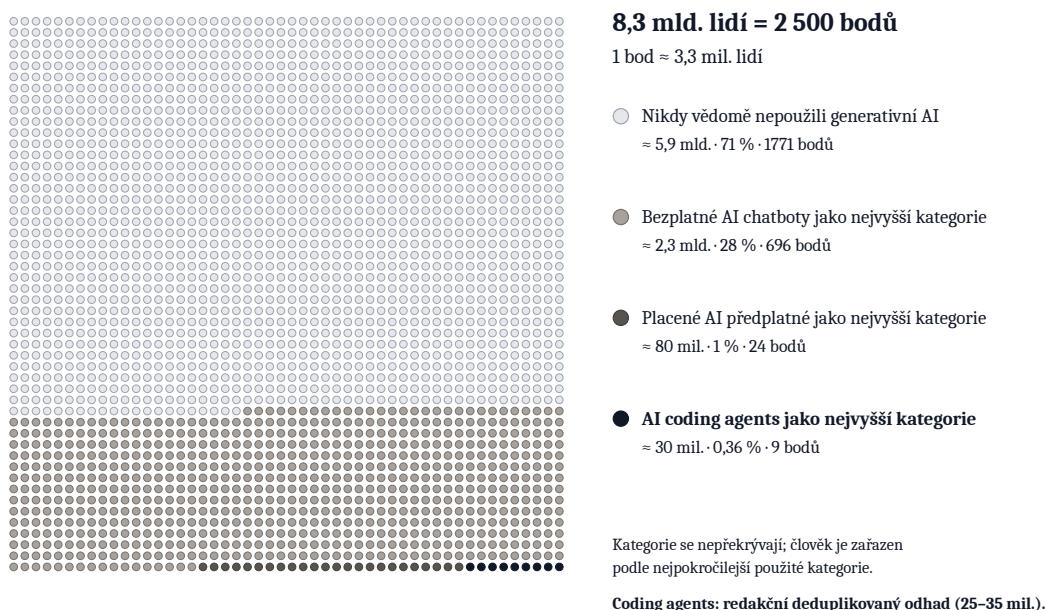
Keywords: coding agents; software factory; AI; Agentic Engineering; harness

Obsah

Prohlášení	2
1 Úvod	5
1.1 Cíl, výzkumná otázka a vymezení	5
1.2 Terminologie	7
2 Teoretická část	8
2.1 Jazykový model v agentním systému	8
2.2 Agent a harness	9
2.2.1 Context window a kompakce	9
2.2.2 Smyčka	9
2.3 Agentické inženýrství	11
2.3.1 Zadání a kontext	11
2.3.2 Dovednosti	11
2.3.3 Orchestrace a lidská integrace	12
2.4 Softwareová továrna	14
2.4.1 Co posunuli agenti	14
3 Praktická část	17
3.1 Metodika	17
3.2 Architektura produkčního běhu	18
3.3 Interpretace a plánování požadavku	20
3.4 Implementace a automatická revize	21
3.5 Zpětná vazba, schválení a úklid	22
4 Výsledky a diskuse	22
4.1 Zjištění	22
4.2 Diskuse	24
4.2.1 Omezení výzkumu	24
5 Závěr	26
Seznam zdrojů	27
Seznam součástí textu	30

1 Úvod

Nástroje založené na velkých jazykových modelech prošly rychlým vývojem: od doplňování kódu při psaní v editoru přes konverzační chatboty až po autonomní agenty, kteří pomocí nástrojů samostatně provádějí změny a spouštějí příkazy v běhovém prostředí (1; 2; 3; 4; 5). S rostoucími schopnostmi modelů roste i jejich adopce, avšak většina uživatelů nemá představu, čeho jsou tyto nástroje skutečně schopny. Většina populace se s generativní AI setkává jenom na povrchu prostřednictvím chatbotů, a to prostřednictvím bezplatného, tedy omezeného, přístupu. Odhad zdroje klade počet uživatelů chatbotů na 28% populace zatímco pravidelné užití AI coding agents pouze na 0,36 % světové populace (6); Obrázek 1 tyto dvě skupiny odlišuje.



Obrázek 1: Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravidelných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6). Aby mohl agent samostatně pracovat na projektu, nestačí pouhé generování odpovědí. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění příkazů, uchování stavu mezi jednotlivými kroky a vymezený bod, v němž člověk rozhodne o přijetí výsledku (7; 8).

Význam klíčových pojmů, které se v českém prostředí neustálily, je zvlášť sezbrán v Kapitola 1.2.

1.1 Cíl, výzkumná otázka a vymezení

Cílem práce je uvést čtenáře do problematiky agentického umělé inteligence: jak vznikla, co jí je, jak se dnes používá a jaké postupy umožňují předávat jí část práce ve vývoji softwaru. Práce vymezuje agentické inženýrství jako soubor těchto postupů a ukazuje, že jeho jádrem není schopnost modelu, ale návrh systému, v němž model pracuje. Druhou polovinou cíle je

praktická demonstrace: postavit co nejjednodušší provozuschopnou softwareovou továrnu, která zobrazí právě ty části postupu, které jsou v dnešní praxi rozhodující.

Z toho vychází výzkumná otázka, na kterou práce odpovídá: *je praktická autonomie ve vývoji softwaru dosažitelná také tehdy, když autor nevlastní smyčku agenta?* Tomu odpovídá hypotéza, že praktická autonomie je vlastností systému a nikoli agenta, a je tedy dosažitelná kombinací trvalého stavu uloženého mimo model, explicitních lidských bran a oddělené kontroly výstupu, aniž by bylo nutné psát si vlastní harness. Hypotéza se ověřuje v praktické části a z výsledku tohoto ověření práce vychází.

Praktická část popisuje DarkFactory, jednoduchou produkční pipeline pro AI-asistovaný softwarový vývoj. GitHub je v ní vývojovým prostředím, GitHub Actions zajišťuje běhy, Docker kontejner odděluje prostředí, Github Issues slouží jako plánovací/stavový systém a pythonovský řadič ve spolupráci s produkčními harnessy (`claude`, `agy`, `codex`, `opencode`) realizuje interpretaci, plánování, implementaci, revizi a integraci změn. Jejich skutečná rozhraní uvádí Kapitola 3.2.

Vymezení práce je záměrné a úzké: popisuje se pouze počáteční implementace této pipeline, v níž je smyčka agenta provedena cizím nástrojem. Její rozsah odpovídá zadání a slouží jako základ, na němž navazuje práce následující.

1.2 Terminologie

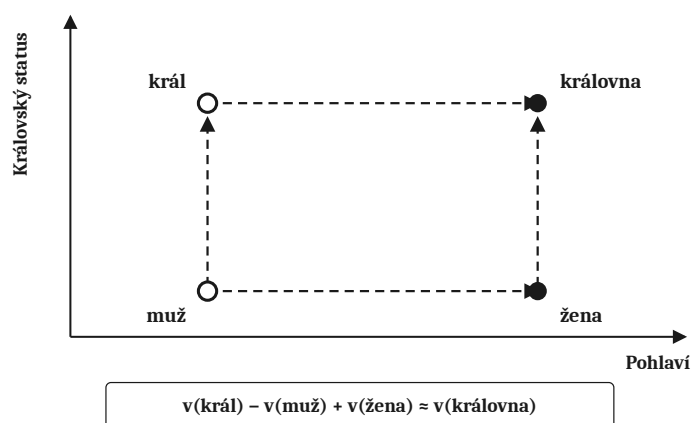
Anglické **agentic** nemá v češtině ustálený překlad; pro potřeby této práce je mu přiřazeno tvořené přídavné jméno **agentické**. Rozdíl mezi dvěma výrazy, které se v běžném užívání zaměňují, je přitom zásadní. **Agentní** označuje vlastnost agenta, tedy toho, kdo jedná; **agentický** označuje vlastnost systému, tedy toho, kdo je **schopen** jednat. Jazykový základ tomu odpovídá: anglické **agentic** je odvozeno od podstatného jména **agent** příponou **-ic** a znamená mající schopnost, prostředky nebo pravomoc jednat, nikoli samo jednajícího (9). V této práci je proto **agentické** užíváno výhradně v druhém smyslu, a **agentické inženýrství (Agentic Engineering)** tak znamená soubor postupů, jimiž se staví systémy s takovým chováním, nikoli stavbu samotných agentů (10). Odtud také, že celou prací prostupuje jedna otázka: je-li praktická autonomie vlastností systému, má v ní poslední slovo návrh prostředí, v němž model pracuje, nebo samotný model.

2 Teoretická část

2.1 Jazykový model v agentním systému

Současný jazykový model stojí na jedné architektuře. V roce 2017 představil Google v příspěvku **Attention Is All You Need** novou neuronovou architekturu **Transformer**, která místo zpracovávání tokenů jeden po druhém přiřazuje význam každému tokenu současně se všemi ostatními. Mechanismus, který to umožňuje, se nazývá **attention** (11) a je dodnes používán i v pozdějších generacích modelů (12). Právě tato schopnost zpracovat celý kontext najednou je to, co umožňuje dnešní požadavky na délku a složitost konverzace. Na této architektuře je založen i jazykový model (**LLM**), který předpovídá další token na základě toho, co je před ním obsaženo v **kontextu**. Při **inferenci** model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy ve vektorovém prostoru. Známým příkladem je vztah mezi vektory slov **král**, **královna**, **muž** a **žena** (13). Tento vztah schematicky znázorňuje Obrázek 2.



Obrázek 2: Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **král**, **královna**, **muž** a **žena** (13).

2.2 Agent a harness

Postupné šíření těchto nástrojů má tři stupně. Nejprve doplňovaly kód v editoru a člověk zůstával tím, kdo jej přijímal a spouští. Potom přišly konverzační chatboty, v nichž model sestavuje odpověď, ale nástroje mu zpravidla nebyly k dispozici, takže i nadále všechno provedl uživatel. Až třetí stupeň, coding agenti, dostal přístup k souborům, příkazům a běhovému prostředí — a tím se poprvé změnila věta, kdo právě jedná.

Tuto hranici lze pojmenovat přesněji: **agent je model plus harness** (14). Harness je všechen kód, konfigurace a vykonávací logika, která není samotným modelem. Model sám o sobě neumí udržet stav mezi kroky, spustit kód, přistupovat k údajům, které se po jeho tréninku změnil, ani připravit si prostředí — a právě to všechno mu musí dodat harness. Hrubý model tedy agentem je teprve tehdy, když mu harness dodá stav, vykonávání nástrojů, zpětné vazby a vynutitelná omezení.

Praktickou hranicí mezi konverzačním chatbotem a agentem je právě míra delegovaného provádění. Samotná **Inference** však nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje, vyřizuje oprávnění a sestavuje prompt z přepisu konverzace. Rozdělení modelu a harnessu je tedy věcí odpovědnosti, nikoli pouhé implementace: první navrhuje, druhý jedná.

2.2.1 Context window a kompakce

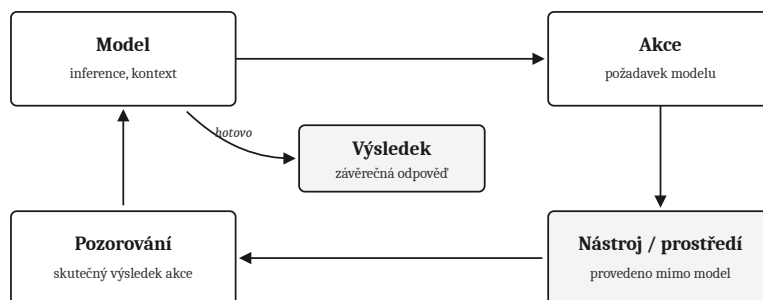
Kontextové okno (context window) tvoří pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat (15). Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** (16).

Kompakce (**compaction**) po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce (16).

2.2.2 Smyčka

Základním mechanismem agentického systému je **agentní smyčka**, tedy konkrétní implementace vzoru **ReAct (Reasoning and Acting)** (17) v daném harnessu: model střídá uvažování s akcí a harness mezi jednotlivými kroky vrací pozorování, čímž z jednotité generace vzniká souvislá konverzace (18). V každém kroku model nejprve zdůvodní, co hodlá udělat, a vyjádří to jako požadavek na nástroj. Harness akci provede v běhovém prostředí a výsledek vrátí modelu jako pozorování; to se připojí k přepisu konverzace a cyklus se opakuje. Smyčka končí teprve tehdy, když model místo dalšího požadavku na nástroj vydá

závěrečnou odpověď. Právě tím se smyčka liší od prosté posloupnosti promptů, v níž by model neměl jak poznat, zda předchozí krok vůbec uspěl; průběh shrnuje Obrázek 3.



Harness přidává každý krok do přepisu konverzace a výsledek nástroje vrací modelu jako nové pozorování.

Obrázek 3: Agentní smyčka ReAct: model navrhne akci, harness ji provede v běhovém prostředí a pozorování se vrací do dalšího kroku. Ukončení nastává, když model místo další akce vydá závěrečnou odpověď (17).

2.3 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) označuje soubor postupů, jimiž se vývoj softwaru pomocí coding agentů stává účinným, kontrolovaným, opakovatelným a škálovatelným (10). Jeho předmětem není samotný agent ani jeho model, ale systém, v němž agent pracuje: zadání, omezení, nástroje, pravidla integrace a odpovědnost člověka. Odtud také jeho popis jako archetypu **agentického inženýra**, jehož přidanou hodnotu už netvoří psaní kódu, ale formulace zadání, řízení agentních běhů a kritické posouzení strojem vytvořených výstupů (19). Mezi hlavní oblasti patří: **Prompt Engineering**, **Context Engineering**, **Harness Engineering**, **Loop Engineering** a **Workflow/Graph Engineering** (7; 16; 20; 21; 22). Cílem je, aby vývojář mohl efektivně a kontrolovaně delegovat dílčí úkoly agentovi, aniž by ztratil přehled o záměru, rozsahu a kvalitě výsledku.

2.3.1 Zadání a kontext

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo **spec-driven development**, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku. Přitom jde o postup používaný i v klasickém vývoji softwaru.

V praxi to znamená, že se plán nevzniká až během práce agenta, ale před ní: jeho znění se nejprve dohodnou s člověkem, zapíše do souboru v repozitáři a teprve potom se agent pustí do změn. Takový plánový soubor je zároveň stav, na který se lze vracet při dalším běhu, a harness ho na začátku běhu připomíná agentu jako použitelnou pomůcku (14). Rozdíl mezi plánem, který si agent sepsal sám, a plánem, který člověk schválil, je přitom právě rozdílem mezi návrhem a oprávněním měnit kód.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku (20). **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici (16).

2.3.2 Dovednosti

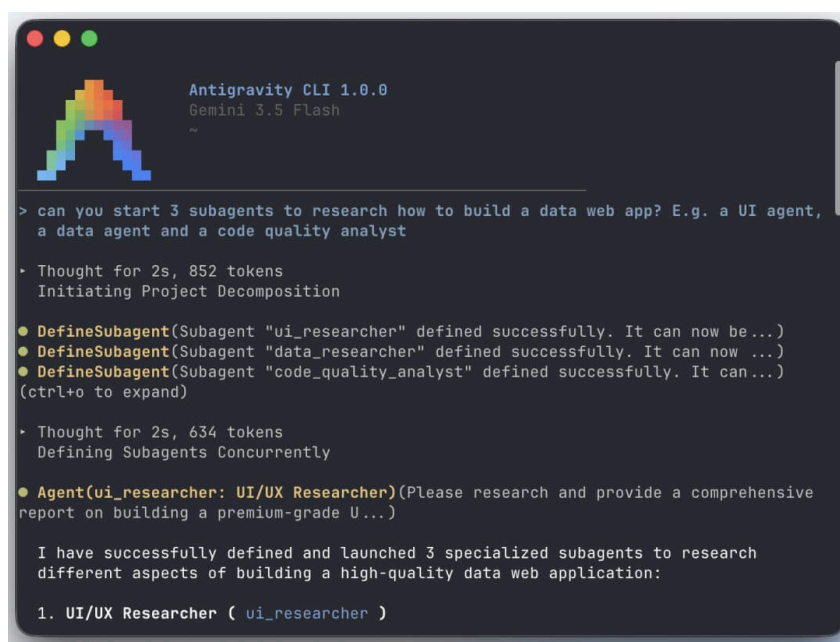
Zadání určuje, co má agent udělat; dovednosti určují, co je mu vůbec umožněno udělat. Druhou skupinu zajišťuje harness, a je proto vhodné ji od zadání oddělit: změna dovedností nemění úkol, změna zadání nemění oprávnění. Rozdělení není vlastní: odpovídá běžnému výčtu součástí harnessu, jak jej uvádí (14) — systémové prompty, nástroje a skills a MCP včetně jejich popisů, doprovodná infrastruktura pro souborový systém, sandbox a prohlížeč, orchestrační logika pro zakládání subagentů, předávání a směřování modelu, a hooky nebo middleware pro deterministický běh.

Nástroje umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy; **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy (23). **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu (24). **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server (25). Instrukce lze uchovat ve standardizovaném souboru `AGENTS.md` (26) přímo v repozitáři (Anthropic ojedinele využívá `CLAUDE.md`). Skilly, skripty a hooky lze uchovat pod složkou `.agents/` v projektu nebo v konfigurační složce harnessu.

Tato dělba má přímý důsledek pro praxi. Oprávnění udělená nástroji a rozsah zadání se dají omezovat nezávisle: lze agentovi odebrat právo měnit cokoli mimo vyjmenované soubory, aniž by se změnil úkol, a lze zúžit úkol, aniž by mu přibyla schopnost. Kontrola, která by tyto dvě roviny zaměnila, by pak nedokázala říct, zda selhal úkol, nebo oprávnění.

2.3.3 Orchestrace a lidská integrace

Pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat, rozsáhlou úlohu lze rozdělit do více agentních běhů. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek. Nezávislé podúlohy mohou zpracovat paralelní pracovníci. **Workflow graph** předem určuje závislosti, pořadí a větvení fází (22). **Agent Swarm** dynamicky rozkládá úlohu na heterogenní podproblémy a spouští specializované agenty paralelně pod řízením orchestrátoru (27). První dva vzory jsou běžně dostupné i v komerčních CLI harnessích, jak ukazuje Obrázek 4.



```
AntigraVity CLI 1.0.0
Gemini 3.5 Flash

> can you start 3 subagents to research how to build a data web app? E.g. a UI agent,
a data agent and a code quality analyst

• Thought for 2s, 852 tokens
  Initiating Project Decomposition

• DefineSubagent(Subagent "ui_researcher" defined successfully. It can now be ...)
• DefineSubagent(Subagent "data_researcher" defined successfully. It can now ...)
• DefineSubagent(Subagent "code_quality_analyst" defined successfully. It can...)
  (ctrl+o to expand)

• Thought for 2s, 634 tokens
  Defining Subagents Concurrently

• Agent(ui_researcher: UI/UX Researcher)(Please research and provide a comprehensive
report on building a premium-grade U...)

I have successfully defined and launched 3 specialized subagents to research
different aspects of building a high-quality data web application:

1. UI/UX Researcher ( ui_researcher )
```

Obrázek 4: Koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem. Převzato z dokumentace produktu jako ilustrace rozhraní, nikoli jako záznam běhu popsané pipeline (28).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol. Orchestrace je však nezbytná právě pro rozsáhlé úlohy, které se do jednoho kontextového okna nevejdou. Spoléhat se v takovém případě na kompakci by vedlo k tomu, že se ztrácí kontext, na kterém celé zadání stojí, a agent by navíc nebyl schopen pracovat na jednotlivých částech, které na sebe bezprostředně navazují.

Goal loop označuje nadřazenou řídicí smyčku: po dílčím dokončení ReAct smyčky harness porovná pozorovaný stav s cílem a podmínkami přijetí a podle výsledku běh ukončí, nebo zahájí další iteraci či změnu strategie. (17).

human-in-the-loop (HITL) označuje bod kde se do automatizované smyčky doplní kontrolní brány, v nichž je vyžadováno explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak může ponechat agentovi ohraničenou implementační práci a současně si zachovat odpovědnost za záměr a integraci.

2.4 Softwareová továrna

Pojem softwareová továrna je starší než generativní umělá inteligence o přibližně padesát let. Zavedl jej Robert Bemer, který pojem vymyslel a definoval jeho povahu ještě v době, kdy ve společnosti General Electric řídil integraci systémového a softwarového inženýrství (29). Výchozí úvahou bylo, že pokud jsou počítače užitečné pro celou třídu úloh, měly by být užitečné i k výrobě vlastních systémů: „Some manufacturers are now building a software factory, or an environment residing upon a computer within which all software production takes place.“ (30) Návrh vycházel z porovnání programování s tovární výrobou a jeho ambicí byla industrializace opakujících se kroků.

Návrh se dostal i na konferenci NATO o softwarovém inženýrství v Garmischi v říjnu 1968, první konferenci, která se tímto tématem zabývala (31). Její zpráva však pojem softwarové továrny nezavedla — popsala softwarovou krizi, na niž Bemer navazoval (32). O jeho návrhu se na konferenci mluvilo jako o tom nejambicióznějším přístupu, jaký byl představen: „The most ambitious approach to this problem was that described by Bob Bemer, who envisaged the use of a ‚machine-controlled production environment, or software factory‘, which would provide an environment for all program construction, check-out and usage.“ (31)

Návrh se v následujících desetiletích ustálil v konkrétní podobě. Michael Cusumano popisuje factory concepts and practices, které zavedla Hitachi, Toshiba, NEC a Fujitsu: standardizaci vývojových metod, systematickou znovuvyužitelnost komponent, dělbu práce a kontrolu kvality, to vše za účelem přechodu od řemeslné práce k opakovatelnému procesu (33). Podstatné je, co v tomto pojetí zůstává na člověku: továrna automatizuje provádění, nikoli rozhodování o tom, co se vyrábí.

2.4.1 Co posunuli agenti

Agenti posunuli hranici, za kterou továrna končila. Zatímco klasická softwareová továrna automatizovala kroky, jejichž postup byl předem dán, agent vykoná i krok, jehož postup předem dán není: zadání přeformuluje, plán sestaví, chybu, kterou sám způsobí, opraví a rozhodne, zda je na řadě další nástroj. Automatizovatelná část se tím posouvá z provádění na úsudek, zatímco člověk zůstává tam, kde byl v každé dosavadní továrně — u určení, co je správné a co smí do výroby vstoupit.

Právě proto se pojem vrací do užívání. Současné popisy této architektury ji označují právě jako software factory: systém, v němž agenti nepřetržitě nacházejí práci, plánují ji, implementují, revidují a předávají výsledek člověku (34; 35). Popisovaný průběh — událost spustí plánování, implementace probíhá na izolované větvi, revizi provádí oddělený krok, sloučení zůstává lidskou bránou — odpovídá tomu, co popisují i systémy postavené mimo tuto práci. Odlišnost je přesto jednoznačná a je vhodné ji vyslovit. Společnost Guild, která takový systém provozuje a sama jej takto označuje, uvádí, že 34 % sloučených pull requestů vzniká

autonomně, 56 % oprav kódu pochází z její factory a 91 % jejích sloučených pull requestů nevyžaduje zásah inženýra (36). Jde ovšem o vlastní měření, nikoli o nezávislé ověření, a oproti popsanému stavu má architektura jednu zásadní odlišnost: plánování, implementaci a revizi u nich zajišťují oddělení agenti, zatímco v této práci jde o tytéž role v jediném harnessu, lišícím se pouze zadáním. Tvrzení o tom, že se tento průběh dnes stává běžnou praxí, lze opřít o průmyslové zprávy o šíření agentů do výroby (37). Z toho plyne i upřesnění, co práce sama přidává. Popsaná pipeline nevymýšlí softwareovou továrnu znovu; naopak se snaží ukázat, že i bez vlastního harnessu, jen s volenou sadou cizích nástrojů, lze její provozní podstaty dosáhnout. Tvrzení, že právě tato část je vlastním přínosem práce, je tedy formulováno jako demonstrace, nikoli jako novost.

PRAKTICKÁ ČÁST

DarkFactory

3 Praktická část

3.1 Metodika

Praktická část implementuje záměrně jednoduchou produkční pipeline, v níž je vývojovým prostředím přímo GitHub a harness produkční coding agent. Cílem je ukázat, jak lze spojit události GitHubu, automatizované plánování, izolovanou práci v kontejneru a lidskou integraci do jednoho opakovatelného procesu.

Předmětem analýzy jsou čtyři navazující vrstvy:

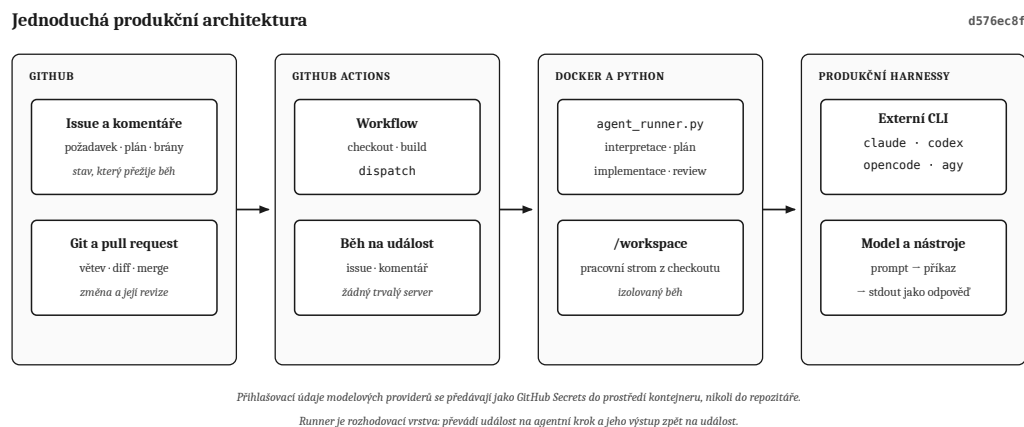
1. **GitHub jako zdroj pravdy:** issue a jeho komentáře uchovávají požadavek, schválení, plán a zpětnou vazbu; větev, commit a pull request uchovávají změnu a její průběžnou revizi (38; 39).
2. **GitHub Actions jako výpočetní prostředí:** jednotlivé události spouštějí krátké workflow, které provede checkout pracovní kopie repozitáře, sestaví obraz, spustí agenta a následnou integraci provede.
3. **Python a Docker jako izolační vrstva:** workflow předá událost a pracovní strom pythonovskému runneru v kontejneru; runner volá harness a předává mu nástroje, přihlašovací údaje a stav úlohy.
4. **Review smyčka jako podmínka integrace:** automatická revize diffu, opravy a další revize se opakují do té doby, než se uzavře poslední nález; teprve poté je pull request předán člověku.

Metodika sama se řídí tvrzením práce. Pythonovský runner, o kterém tato část pojednává, byl napsán co nejjednodušeji pomocí produkčních coding agentů, tedy právě těm nástrojům, které následně popisuje. Cílem nebyla stavba co nejucelenějšího systému, ale co nejmenší uzavřená smyčka, na níž lze ukázat, jak se navzájem ovlivňují agent, harness a lidský recenzent. Pozornost je proto soustředěna na tuto interakci a na její členění, nikoli na šíři nabízených funkcí.

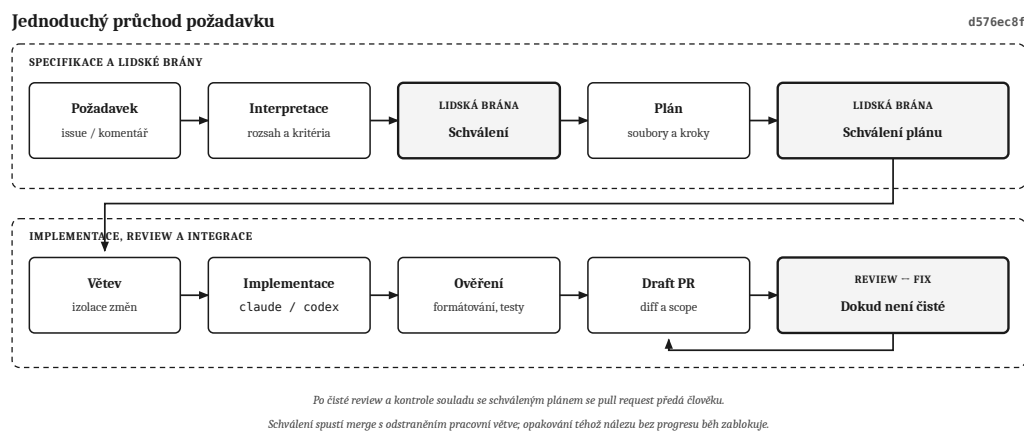
Zjednodušení je záměrné a má svou cenu. Systém bez vlastního stavového serveru, bez databáze a bez trvalého běžícího procesu je na první pohled méně schopný než plnohodnotná platforma, oproti níž je však reprodukovatelný, dohledatelný a jehož každé rozhodnutí zůstává vidět v issue, commitu nebo průběhu kontroly. Zároveň umožňuje zvyšovat složitost postupně: každý nový prvek pipeline byl přidán až poté, co bylo na konkrétní úloze zřejmé, že současná podoba nestačí. Samotná pipeline se tak stala nástrojem, kterým byla psána i další její část.

3.2 Architektura produkčního běhu

Pro základní průchod nepotřebuje DarkFactory nic, co by muselo běžet trvale na vlastním počítači. Všechno, co pipeline potřebuje, už existuje: GitHub slouží jako rozhraní i jako trvalý stavový systém a každá práce agenta probíhá jako izolovaný běh v GitHub Actions. Členění do čtyř vrstev znázorňuje Obrázek 5 a pořadí jednotlivých kroků Obrázek 6.



Obrázek 5: Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy `claude`, `codex`, `opencode` a `agy` zajišťují model, nástroje a pozorování (40).



Obrázek 6: Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka pokračuje do vyřešení nálezů (40).

Workflow `agent.yml` reaguje na otevření issue, nový komentář, review komentář, `repository_dispatch` nebo ruční spuštění. Podmínka na úrovni jobu ověřuje, zda je agent pro repozitář povolen, a filtruje automatické komentáře, aby vlastní výstup pipeline nevytvářel nové události. Po volbě cílového repozitáře workflow provede checkout jeho pracovní kopie, sestaví obraz podle `docker/Dockerfile.agent` a spustí příkaz `dispatch` v kontejneru.

Pythonový runner není náhradou harnessu. Je rozhodovací a integrační vrstvou, která převádí událost GitHubu na konkrétní agentní krok, zpracovává jeho výstup a vyvolává další

událost. Modelové kroky jsou přitom stále prováděny harnessem nad explicitně předaným pracovním stromem (40).

Každý harness je tak definován deklarativně: binár, způsob, jak se z promptu sestaví příkazová řádka, a způsob přihlášení. Přidání harnessu je tak změna dat, nikoli kódu (40).

Přihlašovací údaje se neukládají do repozitáře. GitHub App nebo jiný autorizovaný token se používá pro checkout, issue, pull requesty a push; přihlašovací údaje modelových providerů jsou předány workflow jako GitHub Secrets a následně prostředím kontejneru. Tím pipeline odděluje své automatizační oprávnění od přihlašovacího materiálu agenta a umožňuje změnit poskytovatele bez změny pracovního stromu (40).

3.3 Interpretace a plánování požadavku

Výchozím bodem je issue s formulovaným požadavkem. Požadavek se však nevytváří jako volný text: repozitář nabízí předlohu, která vyžaduje vyplnit několik polí, a právě ta určují, co pipeline dostane. Tabulka 1 uvádí předlohu i vyplněný požadavek, který odpovídá jednomu běhu popsanému v této kapitole.

Pole předlohy	Povinné	Vyplněný požadavek, který spustil běh
Verbatim User Request	ano	„Když review opakovaně nachází to samé, běh se zablokuje. Chtěl bych, aby se po třech neúspěších zastavil a napsal to výslovně do issue.“
Area / Component	ano	<code>ci</code> — GitHub Actions, kontejner, runner
Request Type	ano	<code>feat</code>
Acceptance Criteria	ne	☐ Běh skončí po třetím opakování stejného nálezu. ☐ Do issue vznikne komentář s počtem pokusů. ☐ Test pokrývá nové ukončení. ☐ <code>bun run check</code> projde bez chyb.
Additional Context	ne	<code>block on repeated finding</code> — tři po sobě jdoucí běhy skončily stavem <code>blocked</code> bez změny nálezu.

Tabulka 1: Předloha požadavku a příklad jejího vyplnění. Runner předává modelu titulek a tělo issue; pole předlohy tedy určují, co se do zadání dostane (40).

Příklad je záměrně malý a měřitelný: rozsah se vejde do jednoho kontextového okna, podmínky dokončení jsou pozorovatelné a lze je přepsat na testy. To nejsou náročky pipeline, ale podmínky, za kterých je nasazení vůbec smysluplné — a právě proto je předloha součástí popisu, nikoli jen volný text v issue.

Runner načte jeho titulek a text, vyžádá si od modelu interpretaci a zapíše výsledek jako komentář. Komentář má oddělit doslovné shrnutí požadavku, architektonický rozsah a návrh verifikace. Tím se z chatové odpovědi stane zkontrolovatelný návrh, který lze před dalším během přijmout nebo opravit.

Po schválení interpretace je workflow spuštěno znovu. Model nyní dostane schválený požadavek a sestaví implementační plán, ve kterém uvádí očekávané změny, soubory nebo oblasti repozitáře a kroky ověření. Plán se opět zobrazí v issue. Schválení je tak explicitní bránou: samotná schopnost agenta plán vytvořit neznamena oprávnění měnit kód.

Zpětná vazba člověka není součástí nového vývoje od začátku. Komentář se změnou nebo odmítnutím se předá zpět interpretaci nebo plánování, takže se opravuje rozhodnutí před vytvořením pracovní větve. Tento jednoduchý model odděluje porozumění zadání, plánování a vlastní implementaci bez potřeby složitého grafového orchestrátoru (40).

3.4 Implementace a automatická revize

Po schválení plánu runner vytvoří nebo načte pracovní větev odvozenou z výchozí větve. Implementační instrukce obsahuje schválený plán, omezení na jeho rozsah a pravidla pro testy a dokumentaci. Harness následně může procházet repozitář, upravovat soubory a používat nástroje nad `/workspace`; změny zůstávají izolované mimo výchozí větev.

Po implementaci runner spustí dostupné formátovací nástroje a deklarované testovací sady. Při neúspěchu předá výstup kontroly agentnímu kroku `fix`, který má opravit chybu bez opuštění schváleného rozsahu. Následně runner vytvoří commit, odešle větev a otevře draft pull request. Tím se oddělí samotná změna od jejího posouzení: implementace může být dokončena, ale pull request ještě není připraven k merge (40).

Na draft pull requestu začíná automatická review smyčka. Každá iterace načte aktuální větev a diff a předá jej modelové revizi; souběžně běží průběžná integrace, tedy formátovací nástroje a testovací sady, jejichž výsledek je návratový kód. Pokud review najde chyby, chybějící testy, nevhodný rozsah nebo jiný problém, runner zveřejní nález a spustí nový GitHub Actions běh s fází opravy. Oprava změní větev a spustí další review. Smyčka pokračuje, dokud review nevrátí žádný akční nález; opakovaný stejný nález bez progresu se naopak označí jako zablokovaný stav (40).

Po čisté review ještě proběhne kontrola souladu výsledného diffu se schváleným plánem. Ani ta však není porovnáním sad souborů: je to druhý dotaz modelu, tentokrát na shodu s textem plánu. Teprve když oba dotazy vyjdou bez nálezu, je draft pull request označen jako připravený k lidské revizi. Tento krok je důležitý, protože automatická revize může skončit bez nálezu, aniž by sama zaručila, že implementace odpovídá původnímu zadání.

3.5 Zpětná vazba, schválení a úklid

Po otevření pro lidskou revizi může člověk použít schvalovací workflow nebo zpětnou vazbu na pull requestu. Změnový požadavek zachycuje runner jako opravný běh na stejné větvi. Agent obdrží plán, konkrétní feedback a aktuální kontext větve, provede úpravu, uloží ji jako commit a znovu spustí automatickou review smyčku. Tím se zpětná vazba nevrací do schváleného plánu ani nevyžaduje nový počátek celého požadavku; zachovává se pouze řetězec revizí na jednom pull requestu (40).

Schválení pull requestu zpracovává samostatný workflow. Po ověření, že akci provedl autor issue nebo oprávněný člen repozitáře, převede draft na ready stav, zkontroluje požadované schválení a zapne merge. Při úspěchu se použije `--delete-branch`, takže se pracovní větve po sloučení odstraní. Integrace tedy nekončí pouhým otevřením pull requestu: zahrnuje přechod do ready, splnění ochrany větve, merge a bezpečné odstranění větve (39; 40).

Tato architektura je záměrně jednoduchá. GitHub poskytuje události, schválení, uložení změn a průběh kontroly. GitHub Actions poskytuje výpočet. Docker odděluje běh, Python koordinuje, harness vykonává agentní práci. Složitější služby, trvalý stavový server a více souběžných agentů jsou záměrně mimo základní návrh. Výhodou je snadná reprodukovatelnost a viditelnost každého rozhodnutí. Výhodou je zároveň závislost na dostupnosti GitHubu a na kvalitě promptu, modelu a pravidel repozitáře, kterou samotná automatizace neodstraňuje.

Směr vývoje, který práce záměrně nepopisuje, je vymezen v Kapitola 1.1. Zde je tedy přesnější dodat, kde popsán průchod končí: je popsán stav, v němž pythonovský runner volal cizí produkční CLI (40). Popsaná architektura je tudíž mezníkem, nikoli vyvrcholením, a její popis nenahrazuje posouzení této meze, které patří do Kapitola 4.2. Projekt (41) se od tohoto stavu posunul dál; kam se posunul a proč, je předmětem práce následující, a zde to záměrně není rozvedeno.

4 Výsledky a diskuse

4.1 Zjištění

První zjištění se týká volby vývojového prostředí. Požadavek, jeho schválení, plán, zpětná vazba i výsledná revize zůstávají v GitHubu, takže jednotlivé běhy nemusí sdílet vlastní databázi ani trvalý proces. GitHub Actions přijme událost, připraví pracovní kopii repozitáře a kontejner a následně spustí pythonovský runner. Runner předá práci harnessu a výsledek převede zpět do issue, větve nebo pull requestu (40).

Druhé zjištění potvrzuje oddělení lidského rozhodnutí od modelového návrhu. Issue nejprve vyvolá interpretaci, následně plán a teprve po schválení obou kroků se vytvoří pracovní větve a spustí implementace. Tím pipeline nevytváří změny v hlavní větvi pouze na základě samotného návrhu agenta (40).

Třetí zjištění se týká průběhu revize. Implementace je nejprve odevzdána jako draft pull request. Automatická kontrola nejprve porovná změněné soubory se schváleným plánem a poté předá diff modelové review. Nalezené problémy spouštějí samostatný běh opravy, po němž následuje nová kontrola. Teprve čistá review a kontrola souladu s plánem otevřou pull request pro lidskou revizi (40).

Čtvrté zjištění se týká zpětné vazby a integrace. Změnový požadavek na pull requestu spouští opravu na stejné větvi a po pushi znovu vstupuje do review smyčky. Schválení pull requestu naopak spouští merge s odstraněním větve. Pipeline tedy nekončí automatickým vytvořením kódu, ale přechází do explicitního lidského schválení a následné integrace (40).

4.2 Diskuse

Zjištění lze interpretovat jako konkrétní podobu Agentického inženýrství: praktická autonomie není vlastností samotného modelu, ale výsledkem návrhu prostředí, v němž model pracuje. GitHub poskytuje trvalý kontext a lidskou odpovědnost, Python určuje přechody mezi kroky, Docker omezuje běhové prostředí a harness propojuje model s nástroji a pozorováními. Tato dělba odpovědnosti je v souladu s principy efektivního a kontrolovaného systému popsány v teoretické části (7; 8; 40).

Review smyčka ukazuje výhodu a zároveň omezení průběžné kontroly. Testy a formátovací nástroje poskytují zpětnou vazbu, kterou lze opakovat, ale ani druhá brána — kontrola souladu s plánem — deterministická není: jde o druhý dotaz modelu, a je tedy stejně pravděpodobnostní jako review, které jí předchází. Opakované iterace zvyšují počet příležitostí k nálezům, nikoli záruku konečné správnosti. Zablokování při opakování stejného nálezů je proto rozumnou ochranou před nekonečným během, nikoli důkazem vyřešení problému (40).

Jednoduchost návrhu má také ekonomickou a epistemickou cenu. Nevyžaduje žádnou vlastní infrastrukturu a celý průběh je dohledatelný v issue, commitech, kontrolách a pull requestu. Na druhé straně pipeline závisí na dostupnosti GitHubu, kvalitě issue, schopnosti modelu pracovat s nástroji a pravidlech, která definují akceptační podmínky. Jednoduchá architektura tedy usnadňuje reprodukci postupu, ale nenahrazuje odpovědnost člověka za architekturu, bezpečnost a přijetí výsledku (40).

Opakovatelná a pravděpodobnostní pozorování se proto doplňují, nenahrazují. Formátovací nástroje a testy poskytují pozorování, které se dá opakovat; review, kontrola souladu s plánem, plánování a opravy interpretují zadání a neočekávané nálezy. Z toho plyne, že obě brány před sloučením nejsou nezávislé: jsou to dva dotazy téhož druhu, a druhý z nich navíc vychází z plánu, jehož text sám napsal model. Tato kombinace odpovídá cíli Agentického inženýrství zvýšit užitečnost delegované práce při zachování explicitních bran, ale její skutečná kvalita zůstává závislá na datech, promptu, modelu a konkrétním repozitáři (40).

4.2.1 Omezení výzkumu

Praktická část má omezení, která je třeba pojmenovat, protože se dotýkají platnosti závěrů. Spočívají ve dvou vrstvách.

První je vlastnictví. Agentickou smyčku v popsané realizaci neautor navrhuje: sestavuje ji cizí harness, která spravuje kontext, volá nástroje a vyřizuje oprávnění. Co autor navrhuje, je vrstva nad ní, tedy stavový stroj, který rozhoduje, kdy se smyčka spustí, co smí agent změnit a kde práce skončí. Odpověď na výzkumnou otázku je tím užší, než kdyby smyčka byla implementována od začátku, a je takto i formulována.

Druhé je rozsah. Zkoumaný stav je záměrně jen počáteční: řadič volá cizí harnessy a všechny agentní kroky vede přes ně. Popisuje tedy míru autonomie dosažitelnou bez vlastního

harnessu, nikoli míru dosažitelnou s ním, a tím záměrně neřeší otázku, kterou řeší práce následující. Tvrzení práce se proto vztahuje k této konfiguraci a je opřeno o konkrétní revizi (40).

Z prvního omezení plyne i obrana závěrů. Harness je v popsané architektuře zaměnitelná závislost, stejně jako databáze nebo runner. Co práce především ukazuje, není volba konkrétního CLI, ale struktura procesu: že požadavek musí projít interpretací, plánem a dvěma lidskými branami dříve, než vznikne větev, a že změna se přebírá jako konkrétní revize na jednom pull requestu, dokud nesplní obě brány. Tato struktura je vůči tomu, kdo vykonává modelové kroky, invariantní. Právě to je obsahem Agentického inženýrství ve smyslu vymezeném v teoretické části, a právě to zůstává platné i tehdy, když se harness, jehož rozhraní je popsáno výše, přestane používat. Hypotéza je tím potvrzena v užší podobě, než byla formulována: autonomie je dosažitelná bez vlastního harnessu, ale její kvalita je omezena kvalitou cizí smyčky, kterou autor neovládá.

5 Závěr

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness (7; 18). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů Agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Implementace pipeline DarkFactory ukázala jednoduchou produkční realizaci těchto postupů. GitHub slouží jako vývojové prostředí a místo trvalého stavu, GitHub Actions jako výpočetní prostředí, Docker kontejner jako izolace a produkční harness pro modelovou práci s nástroji. Požadavek prochází interpretací a plánováním, implementace probíhá na samostatné větvi a změna je předána přes draft pull request a opakovací smyčku review–fix. Po čisté review následuje lidské schválení, merge a odstranění větve.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního provádění kódu k formulaci zadání, schvalování plánu, definování akceptačních podmínek a rozhodování, zda je výsledek připraven k integraci. Práce tedy nepopisuje plnou autonomii agenta, ale kontrolované předání vybrané části softwarového inženýrství.

Zjištění zároveň ukazují meze této realizace. Modelové review může být opakované, ale není deterministickou zárukou správnosti; opakovaný nález bez progresu vede k blokaci.

Cíle práce se podařilo naplnit v rozsahu, který si sama vymezila. Hypotéza byla potvrzena, ovšem v užší podobě, než byla formulována: řízená autonomie je dosažitelná i bez vlastního harnessu, ale její kvalita zůstává omezena kvalitou cizí smyčky, kterou autor neovládá. Současně se potvrdilo, že popsaná struktura procesu je vůči volbě nástroje invariantní, což je tvrzení, z něhož lze prokazatelně vycházet.

Pro další výzkum z toho vyplývají tři směry. První je vlastní harness: popsat architekturu, kterou si tato práce záměrně neřešila, a porovnat její výsledky s výsledky cizích nástrojů na stejných úlohách. Druhý je měření: dosavadní zjištění jsou kvalitativní, chybí jim měření, která by umožnila říci, o kolik se cyklus zkrátil a kde přesně review smyčka nachází nejvíce nálezů. Třetí je přenositelnost: ověřit, zda popsaná struktura platí i pro jiný stavový podklad než issue, což by ukázalo, že je opravdu zásadní, a ne pouze vlastností GitHubu.

V praxi má práce přímý dopad v podobě jednoduchého pravidla: delegovat agentovi až to, co lze zkontrolovat, a ponechat mu vlastní smyčku jen tam, kde je výhodná. Kdo takovou pipeline staví, nemusí začínat vlastním harnessem; začíná stavem uloženým mimo model, dvěma branami před vznikem větve a oddělenou kontrolou výstupu.

Seznam zdrojů

1. GITHUB. *GitHub Copilot code suggestions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. *Asking GitHub Copilot questions in your IDE* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. *Use GitHub Copilot agents* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. *Introducing Codex* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-codex/>
5. OPENAI. *Introducing the Codex app* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. *How Many People Use AI? Facts & Figures 2026* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://www.gradually.ai/en/how-many-people-use-ai/>
7. ANTHROPIC. *Harness design for long-running application development* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
8. ANTHROPIC. *Scaling Managed Agents: Decoupling the brain from the hands* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://www.anthropic.com/engineering/managed-agents>
9. MERRIAM-WEBSTER. *Agentic: Definition & Meaning* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://www.merriam-webster.com/dictionary/agentive>
10. WILLISON, Simon. *What is agentic engineering?* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://simonwillison.net/guides/agentic-engineering-patterns/what-is-agentive-engineering>
11. VASWANI, Ashish, Noam SHAZEER, Niki PARMAR, Jakob USZKOREIT, Llion JONES, Aidan N. GOMEZ, Łukasz KAISER a Illia POLOSUKHIN. Attention Is All You Need. *arXiv preprint arXiv:1706.03762* [online]. 2017. Dostupné z: <https://arxiv.org/abs/1706.03762>
12. BROWN, Tom B., Benjamin MANN, Nick RYDER, Melanie SUBBIAH, Jared KAPLAN, Prafulla DHARIWAL, Arvind NEELAKANTAN, Pranav SHYAM, Girish SASTRY, Amanda ASKELL a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* [online]. 2020, **33**. Dostupné z: <https://arxiv.org/abs/2005.14165>

13. MIKOLOV, Tomas, Wen-tau YIH a Geoffrey ZWEIG. Linguistic Regularities in Continuous Space Word Representations. In: *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* [online]. 2013, s. 746–751. Dostupné z: <https://aclanthology.org/N13-1090/>
14. TRIVEDI, Vivek. *The Anatomy of an Agent Harness* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>
15. LIU, Nelson F., Kevin LIN, John HEWITT, Ashwin PARANJAPE, Michele BEVILACQUA, Fabio PETRONI a Percy LIANG. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* [online]. 2024, **12**, 157–173. Dostupné z: <https://arxiv.org/abs/2307.03172>
16. ANTHROPIC. *Effective context engineering for AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
17. YAO, Shunyu, Jeffrey ZHAO, Dian YU, Nan DU, Izhak SHAFRAN, Karthik NARASIMHAN a Yuan CAO. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* [online]. 2022. Dostupné z: <https://arxiv.org/abs/2210.03629>
18. ANTHROPIC. *Building effective agents* [online]. 2024 [vid. 2026-09-23]. Dostupné z: <https://www.anthropic.com/engineering/building-effective-agents>
19. ALENEZI, Mamdouh. Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era. *arXiv preprint arXiv:2607.29610* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://arxiv.org/abs/2607.29610>
20. OPENAI. *Prompt engineering* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://platform.openai.com/docs/guides/prompt-engineering>
21. OPENAI. *Sandbox agents: Concepts | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
22. OPENAI. *Agent orchestration | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-21]. Dostupné z: https://openai.github.io/openai-agents-python/multi_agent/
23. AGENT SKILLS. *Agent Skills: an open format for extending AI agents* [online]. 2025 [vid. 2026-09-22]. Dostupné z: <https://agentskills.io/home>
24. OPENAI. *Lifecycle | OpenAI Agents SDK* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
25. MODEL CONTEXT PROTOCOL. *Model Context Protocol 2026-07-28 specification: server features* [online]. 2026 [vid. 2026-09-22]. Dostupné z: <https://modelcontextprotocol.io/specification/2026-07-28/server>

26. AGENTIC AI FOUNDATION. *AGENTS.md* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://agents.md/>
27. KIMI TEAM. Kimi K2.5: Visual Agentic Intelligence. *arXiv preprint arXiv:2602.02276* [online]. 2026 [vid. 2026-09-23]. Dostupné z: <https://arxiv.org/abs/2602.02276>
28. GOOGLE. *Antigravity CLI* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://antigravity.google/product/antigravity-cli>
29. COMPUTER HISTORY MUSEUM. *Robert W. Bemer* [online]. [vid. 2026-09-27]. Dostupné z: <https://history.computer.org/bemer.html>
30. BEMER, Robert W. *Published Papers, 1951–1999* [online]. 2021 [vid. 2026-09-27]. Dostupné z: <https://archive.computerhistory.org/resources/access/text/2021/04/102785423-05-03-acc.pdf>
31. RANDELL, Brian. *Software Engineering in 1968* [online]. 1968 [vid. 2026-09-27]. Dostupné z: <http://homepages.cs.ncl.ac.uk/brian.randell/PapersInProceedings/349.pdf>
32. NATO. *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 October 1968*. Brussels: NATO, Scientific Affairs Division, 1969.
33. CUSUMANO, Michael A. Factory Concepts and Practices in Software Development. *Annals of the History of Computing* [online]. 1991, **13**(1), 3–32 [vid. 2026-09-27]. Dostupné z: <https://doi.org/10.1109/MAHC.1991.10004>
34. FACTORY.AI. *Factory 2.0: From coding agents to software factories* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://factory.ai/news/software-factory>
35. BCG PLATINION. *The Agentic Software Factory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.bcgplatinion.com/insights/the-agentic-software-factory>
36. GUILD.AI. *Introducing Guild Software Factory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://www.guild.ai/blog/product/introducing-guild-software-factory>
37. ANTHROPIC. *The 2026 State of AI Agents Report* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://resources.anthropic.com/hubfs/The%202026%20State%20of%20AI%20Agents%20Report.pdf>
38. GITHUB. *Branches | GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/reference/branches>
39. GITHUB. *About pull requests | GitHub Docs* [online]. 2026 [vid. 2026-09-21]. Dostupné z: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
40. MARIUS, Patrik. *DarkFactory commit d576ec8f* [online]. 2026 [vid. 2026-09-26]. Dostupné z: <https://github.com/marius-patrik/DarkFactory/commit/d576ec8fdb78a654bffc9d0da165aa81a97b673>
41. MARIUS, Patrik. *DarkFactory* [online]. 2026 [vid. 2026-09-27]. Dostupné z: <https://github.com/marius-patrik/DarkFactory>

Seznam součástí textu

Obrázek 1	Odhad rozdělení uživatelů generativní AI podle typu. Hodnota 30 milionů pravidelných uživatelů coding agents, přibližně 0,36 % světové populace, je redakční střed odhadovaného rozmezí 25–35 milionů (6).....	5
Obrázek 2	Ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov král , královna , muž a žena (13).....	8
Obrázek 3	Agentní smyčka ReAct: model navrhne akci, harness ji provede v běhovém prostředí a pozorování se vrací do dalšího kroku. Ukončení nastává, když model místo další akce vydá závěrečnou odpověď (17).....	10
Obrázek 4	Koordinátor definuje tři specializované subagenty a spouští je souběžně; každý běží vlastním kontextem. Převzato z dokumentace produktu jako ilustrace rozhraní, nikoli jako záznam běhu popsané pipeline (28).....	12
Obrázek 5	Architektura: GitHub poskytuje události a vývojový stav, GitHub Actions výpočet, Docker odděluje běh a pythonovský runner převádí událost na agentní krok. Produkční harnessy <code>claude</code> , <code>codex</code> , <code>opencode</code> a <code>agy</code> zajišťují model, nástroje a pozorování (40).....	18
Obrázek 6	Průchod požadavku: implementace a plánování jsou odděleny lidskými bránami, implementace probíhá na větvi a review smyčka pokračuje do vyřešení nálezů (40).....	18
Tabulka 1	Předloha požadavku a příklad jejího vyplnění. Runner předává modelu titulek a tělo issue; pole předlohy tedy určují, co se do zadání dostane (40).....	20